PROJECT PROPOSAL

AI College Assistant

A RAG-Based Conversational Assistant for Campus Information and Academic Support

Submitted by	Naitik
Program	B.Tech, Computer Science Engineering (AI & ML)
Institution	GLA University, Mathura, Uttar Pradesh
Year	Third Year (2024–2028)
Date	4 August 2026

1. Executive Summary

The AI College Assistant is a proposed RAG-based (Retrieval-Augmented Generation) conversational system designed to help students quickly access campus information and academic support through natural-language queries. Instead of navigating multiple college portals, notice boards, and scattered study material, students will be able to ask direct questions — such as fee deadlines, attendance status, or a specific syllabus module — and receive accurate, contextual answers instantly.

The project is designed to be built and hosted entirely on free-tier cloud services, making it a zero-cost, student-executable initiative that can be piloted within the department before a wider rollout.

2. Problem Statement

Students currently rely on fragmented sources for essential academic information:

- Fee deadlines and notices are posted on college portals or physical noticeboards, easy to miss.
- Subject notes are scattered across WhatsApp groups, drives, and printed handouts of varying quality.
- Attendance and eligibility status require manually logging into the ERP and calculating percentages.
- Doubts about syllabus modules often go unanswered outside class hours.

This fragmentation costs students time and leads to missed deadlines and inconsistent exam preparation. A single, always-available assistant addresses this gap directly.

3. Objectives

- To design a conversational assistant that answers academic and administrative queries in natural language.
- To enable students to upload and query their own study material (PDFs, notes) through the assistant.
- To summarize official college notices and circulars into short, readable updates.
- To build the system using entirely free and open-source tools, keeping it viable for student-level execution.
- To create an extensible architecture that can later support live attendance and timetable integration.

4. Proposed Features

The project is scoped into a core MVP (Minimum Viable Product) and a future roadmap, ensuring a working, demonstrable system within the project timeline.

4.1 Core Features (MVP)

Feature	Description
PDF Chat	Students upload subject notes or textbooks (PDF); the assistant answers questions grounded strictly in that document using a retrieval-augmented pipeline.
Notice Summarizer	Official college notices/circulars are condensed into short, plain-language summaries with key dates highlighted.
Notes Q&A	Students ask subject/module-specific questions (e.g. “Explain Module 4”) and receive answers grounded in the uploaded syllabus material.

4.2 Future Roadmap (Post-MVP)

Feature	Description
Assignment Helper	Guided assistance for structuring and reviewing assignments (concept clarification, not answer generation).
Attendance Tracking	Displays attendance percentage and eligibility status, pending integration with the college ERP or manual data export.
Timetable Assistant	Answers schedule-related queries (“What's my next class?”) once timetable data is integrated.

Note: Attendance and timetable features depend on access to institutional data (ERP export or API). These are scoped as future integrations rather than part of the initial MVP, pending approval for data access.

5. Technology Stack

The entire system is built using free-tier and open-source tools, requiring no financial investment — an important constraint for a student-led project.

Frontend	React.js, deployed free on Vercel
Backend	FastAPI (Python), deployed free on Render
AI / LLM Layer	Google Gemini API or Groq API (free tier)
Retrieval (RAG)	sentence-transformers for embeddings, ChromaDB / FAISS for vector search
Database	MongoDB Atlas (free tier, 512 MB)
OCR	Tesseract OCR, for scanned notices and handwritten/printed material

6. System Architecture (Overview)

The system follows a standard Retrieval-Augmented Generation (RAG) pipeline:

- Documents (notes, notices) are ingested and split into text chunks.
- Each chunk is converted into a vector embedding and stored in a vector database.
- On a student query, the most relevant chunks are retrieved based on semantic similarity.

- The retrieved context is passed to the LLM, which generates an answer grounded in that material — reducing hallucination compared to a generic chatbot.

This ensures the assistant answers using actual college/course content rather than generic internet knowledge.

7. Implementation Plan

Phase	Duration	Deliverable
Requirement gathering & architecture design	Week 1–2	Finalized system design, tech stack setup
RAG pipeline development	Week 3–4	Working PDF Chat (document ingestion + Q&A)
Notice Summarizer module	Week 5	Notice upload + summarization feature
Notes Q&A module	Week 6–7	Module-wise Q&A grounded in syllabus content
Frontend integration & testing	Week 8–9	End-to-end working web application
Deployment & documentation	Week 10	Live demo deployment, final report

8. Expected Outcomes

- A working, demo-ready web application answering real student queries from actual course material.
- Reduced time spent searching for notices, deadlines, and notes across scattered sources.
- A reusable, extensible architecture that the department can build on for future ERP integration.
- Hands-on experience across the full AI/ML application stack: RAG, embeddings, vector search, and full-stack deployment.

9. Conclusion

The AI College Assistant addresses a genuine, everyday problem faced by students using a practical, zero-cost technical approach. By scoping the initial build to three well-defined, achievable features (PDF Chat, Notice Summarizer, and Notes Q&A), the project ensures a working deliverable within the proposed timeline, while leaving a clear path for future integration with attendance and timetable systems as institutional data access becomes available.

This proposal is submitted for review and approval to proceed with implementation.